AI Agent Security: From Prompt Injection to Unsafe Tool Execution and Unified Evaluation

Anonymous Author(s)
Draft prepared in the academic-workspace research workflow

***Abstract*—AI agents extend large language models from text generation to action-taking systems that can plan, call tools, read and write memory, execute code, and operate across external environments. This shift changes the central security question from whether a model produces an incorrect answer to whether a compromised or misguided model decision becomes a high-impact external action. This paper synthesizes a focused set of representative works that mark the transition from prompt injection in LLM-integrated applications to broader agent security evaluation. We analyze four anchor papers: *Prompt Injection attack against LLM-integrated Applications*, *ToolEmu*, *AgentDojo*, and *Agent Security Bench (ASB)*. These works jointly show that agent security is not a single benchmark problem, but a composite of goal hijacking, unsafe tool use, privilege amplification, memory and context poisoning, hidden planning manipulation, and governance gaps. We compare their threat models, evaluation targets, and limitations, and argue that future research should move beyond prompt-layer defenses toward integrated treatment of permissions, memory, supply chain, approval design, and post-action accountability.**

***Index Terms*—AI agents, agent security, prompt injection, tool use safety, benchmark, memory poisoning**

I. Introduction

Large language models (LLMs) are increasingly wrapped into agentic systems that can perform planning, invoke external tools, browse websites, execute code, access files, and coordinate multi-step workflows. This design expands usefulness, but it also changes the failure surface. A wrong answer in a chat interface may be recoverable. A wrong action in an agent can leak data, send money, execute a destructive shell command, or persist a poisoned memory state for future tasks.

The literature has already moved in this direction. Early prompt injection studies asked whether adversarial input can override an application's hidden instructions. More recent agent benchmarks ask whether an agent can complete the user's task while resisting malicious instructions embedded in tools, memory, or planning scaffolds. At the same time, another line of work studies whether agents can safely operate in high-stakes environments even without a classical attacker, simply because user requests are ambiguous and the agent is overconfident.

This paper organizes that transition through four representative works:

- *Prompt Injection attack against LLM-integrated Applications* [1];
- *ToolEmu* [2];
- *AgentDojo* [3];
- *Agent Security Bench (ASB)* [4].

The contribution of this paper is not a new benchmark or defense. Instead, it provides a compact, publication-style synthesis that clarifies what each work actually evaluates, where they complement each other, and which research gaps remain open.

II. From Prompt Injection to Agent Security

A. Prompt Injection as the Prehistory of Agent Hijacking

Liu et al. [1] analyze prompt injection in real LLM-integrated applications under a black-box threat model. Their central observation is that successful injection depends on making the target model interpret malicious text as executable intent rather than passive data. This is the conceptual foundation for later agent security work. Once a model is attached to tools and actions, the same instruction–data ambiguity no longer only changes text outputs. It may redirect planning, tool selection, tool parameters, and downstream execution.

The paper's practical contribution is *HOUYI*, a structured attack framework that decomposes payload construction into a framework component, a separator, and a disruptor. The authors report successful exploitation against 31 out of 36 tested LLM-integrated applications, with an overall success rate of 86.1% in their setup [1]. The exact percentages are less important than the structural lesson: injection strength in deployed systems depends on application composition, formatting constraints, and multi-step interaction patterns.

B. AgentDojo: Dynamic Tool Environments Under Attack

AgentDojo [3] shifts the field from static prompt manipulation to dynamic tool-calling environments. Rather than evaluating one-off responses, it evaluates agents operating over stateful applications such as productivity suites, travel booking, banking, and Slack-like environments. The benchmark includes 97 realistic tasks and 629 security test cases.

Its main methodological contribution is that utility and security are jointly grounded in deterministic checks over environment state. This matters because low attack success can be misleading if the agent fails to perform useful work in the first place. AgentDojo explicitly measures benign utility, utility under attack, and targeted attack success rate. The paper reports that current models solve fewer than 66% of tasks in benign settings, while attacks against top-performing agents succeed in fewer than 25% of cases; with some defenses, attack success can drop to around 8%, often at noticeable utility cost [3].

The broader insight is that agent security must be treated as a utility–security tradeoff problem. An "agent" that refuses

to act or cannot complete tasks may look robust while being practically useless.

C. ToolEmu: Unsafe Tool Execution Without a Classical Adversary

ToolEmu [2] studies a different but equally important problem. Its threat model is not primarily malicious prompt injection. Instead, it focuses on high-stakes failures caused by ambiguous instructions, missing constraints, and unsafe autonomy. To make scalable testing practical, ToolEmu uses an LM-emulated sandbox rather than a fully implemented real environment for each tool.

This design allows broad risk discovery across many toolkits and scenarios. ToolEmu reports that 68.8% of failures identified by the framework are validated by humans as realistic risky failures, and that even its safest GPT-4-based agent still fails in 23.9% of test cases [2]. The significance of ToolEmu is that it expands agent security beyond adversarial compromise. In real deployments, severe harm can arise even when no attacker is present, simply because the agent is allowed to act under underspecified conditions.

D. ASB: Unifying Multiple Agent Attack Surfaces

Agent Security Bench (ASB) [4] attempts to unify agent security across multiple operational stages. The benchmark spans 10 scenarios, 10 corresponding agents, more than 400 tools, 27 attack and defense types, and seven metrics. Most importantly, it formalizes attacks not only on user prompts and tool outputs, but also on system prompts, memory retrieval, and planning.

ASB covers direct prompt injection, indirect prompt injection, memory poisoning, Plan-of-Thought backdoor attacks, and mixed attacks. The authors report that mixed attacks achieve the highest average attack success rate at 84.30%, while memory poisoning is substantially lower at 7.92% in their benchmark configuration [4]. They also introduce the Net Resilient Performance (NRP) metric to jointly account for normal task performance and resilience under attack.

ASB is valuable because it makes clear that "agent security" is not synonymous with indirect prompt injection. At the same time, it reveals the difficulty of aggregating heterogeneous attacks under one score: attack realism, maturity, and downstream harm differ significantly by attack type.

III. A Working Taxonomy for AI Agent Security

Synthesizing the four core papers with practitioner guidance from OWASP and NIST [5], [6], [7], [8], we propose a six-part working taxonomy.

A. Goal Hijacking

Goal hijacking occurs when malicious content changes an agent's task priorities, execution order, or intended objective. This category includes classical prompt injection and its agentic variants. The distinguishing feature in agents is that successful hijacking can alter action plans rather than merely text responses.

B. Unsafe Tool Use

Unsafe tool use includes selecting the wrong tool, inferring unsafe parameters, acting on low-confidence assumptions, or executing high-impact commands without sufficient clarification. ToolEmu shows that this failure mode matters even without an active attacker.

C. Privilege Amplification

The same reasoning error has very different consequences depending on agent privileges. A mistaken inference in a read-only chat system may be harmless; the same inference in a code runner, payment workflow, or file operator can cause severe damage. This is why permissions must be treated as part of the threat model rather than an implementation detail.

D. Memory and Context Poisoning

Persistent memory, retrieval databases, prior plans, and context caches may alter future behavior across sessions. ASB includes memory poisoning, but this area still lacks mature, realistic, and widely accepted evaluation settings.

E. Hidden Planning and System Manipulation

Agents often rely on hidden system prompts, planning scaffolds, and internal action formats. ASB's Plan-of-Thought backdoor highlights that attacks can target these hidden structures, not only visible user or tool inputs.

F. Evaluation and Governance Gaps

Benchmarks do not directly answer deployment questions such as when a human approval should be required, how audit trails should be structured, what should be logged, or how a compromised run should be recovered. Governance is therefore part of agent security, not only an organizational add-on.

IV. Benchmark Targets Are Not the Same Problem

One recurring source of confusion is that these papers are often grouped together as "agent benchmarks" even though they answer different questions.

ToolEmu is best understood as a *risk discovery framework*. AgentDojo is best understood as a *dynamic adversarial evaluation environment*. ASB is best understood as a *unified multi-surface benchmark*. Liu et al. provide the *prompt injection precursor model* that explains why later agent attacks are possible in the first place.

Treating them as interchangeable leads to category mistakes. For example, a benchmark optimized for adversarial success rate may miss ordinary but dangerous tool misuse. Conversely, a risk-discovery framework may find severe failures while saying little about attacker adaptation or defense robustness.

V. Research Gaps

Despite rapid progress, the current literature still leaves several important gaps.

TABLE I
REPRESENTATIVE WORKS AND WHAT THEY ARE ACTUALLY BEST AT MEASURING

Work	Primary threat model	Evaluation target	Strength	Main limitation
Liu et al.	Black-box prompt injection in deployed LLM-integrated apps	Attack construction and exploitability	Explains instruction–data ambiguity and systematic payload generation	Not a full agent setting; limited treatment of permissions and memory
ToolEmu	Unsafe autonomy under ambiguous or underspecified requests	Long-tail risk discovery in tool execution	Captures realistic high-stakes failures beyond adversarial injection	LM-emulated environment may diverge from real systems
AgentDojo	Indirect prompt injection in dynamic tool environments	Joint utility and security under attack	Strong treatment of stateful environments and utility–security tradeoff	Focus remains centered on tool-driven prompt injection
ASB	Multi-stage attacks on system prompt, memory, planning, and tools	Unified attack/defense benchmark	Broadest attack-surface framing among the four works	Aggregates heterogeneous attacks whose realism and maturity vary

A. Permissions Are Under-Theorized

Most defenses still focus on prompts or detectors. Fewer works build principled models of minimum permissions, bounded autonomy, staged execution, or action containment. Yet privileges are what turn language-level compromise into operational harm.

B. Memory Security Is Early

Memory poisoning has entered benchmark design, but the field still lacks robust, realistic evaluations for persistent memory, retrieval caches, and cross-session contamination in deployed agent systems.

C. Supply Chain Security Remains Thin

Modern agents depend on tools, skills, plugins, browser automation, workflow templates, and model context protocols. These are not only software dependencies. They are often natural-language dependencies that influence agent planning directly. Current literature under-covers this attack surface.

D. Human Approval Is Underspecified

Many papers assume that a human-in-the-loop mitigates residual risk. In practice, approval design raises difficult questions: when should approval be required, at what granularity, with what explanation, and under what fatigue constraints? This remains largely underdeveloped.

E. Governance Metrics Lag Technical Metrics

Attack success rate, refusal rate, and utility are useful, but they do not capture auditability, rollback quality, incident traceability, or organizational recovery readiness. Deployment-oriented agent security will need these metrics.

VI. CONCLUSION

AI agent security should not be framed as a narrow extension of prompt injection research. It is a broader systems problem created by coupling language models to memory, tools, permissions, execution loops, and organizational workflows. Prompt injection explains the root ambiguity between instructions and data. ToolEmu shows that unsafe autonomy can fail dangerously even without an attacker. AgentDojo demonstrates that utility and robustness must be measured together in dynamic environments. ASB shows that the relevant attack surface spans system prompts, memory, planning, and tools, not only user input.

The next step for the field is not merely another benchmark. It is a tighter connection between technical evaluation and deployment controls: permission design, approval mechanisms, memory integrity, skill and tool supply chain security, and post-action auditability. A credible security story for agents must address all of them.

REFERENCES

[1] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt Injection attack against LLM-integrated Applications," arXiv:2306.05499, 2023.

[2] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *International Conference on Learning Representations (ICLR)*, 2024.

[3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

[4] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents," in *International Conference on Learning Representations (ICLR)*, 2025.

[5] OWASP, "Agentic AI Threats and Mitigations," 2025. [Online]. Available: https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/

[6] OWASP, "OWASP Top 10 for Agentic Applications for 2026," 2025. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[7] OWASP, "OWASP Agentic Skills Top 10," 2026. [Online]. Available: https://owasp.org/www-project-agentic-skills-top-10/

[8] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, 2024.